

UNIVERSIDAD CATÓLICA DE TEMUCO

DEPARTAMENTO DE INGENIERÍA INFORMÁTICA
FACULTAD DE INGENIERÍA

# Evaluación experimental de eficiencia energética y rendimiento en clústeres de Alta Disponibilidad con K3s y MicroK8s

por

**José Ignacio Delpino Muñoz**

Trabajo presentado a la
Facultad de Ingeniería de la Universidad Católica de Temuco
Para optar al título de Ingeniero Civil Informático.

- Temuco, Octubre 2026 -

# Índice

**1. Incorporación de la retroalimentación de la Etapa 1** **1**

**2. Diseño de infraestructura virtualizada y red** **2**
2.1. Topología general . . . 2
2.2. Modo de conectividad de red . . . 2
2.3. Sistema operativo invitado . . . 3
2.4. Dimensionamiento de recursos por nodo . . . 4
2.5. Direccionamiento y acceso redundante al plano de control . . . 5

**3. Diseño de arquitectura de observabilidad** **6**
3.1. Arquitectura general . . . 6
3.2. Medición energética con Scaphandre y RAPL/powercap . . . 6
3.3. Recolección de CPU y RAM con Node Exporter . . . 7
3.4. Almacenamiento y consultas en Prometheus . . . 7
3.5. Aplicación de referencia . . . 8
3.6. Mecanismo de exposición: reutilización de kube-vip . . . 9
3.7. Instrumentación de rendimiento con k6 . . . 9
3.8. Observación de resiliencia y sincronización temporal . . . 9

**4. Diseño del protocolo experimental** **10**
4.1. Escenarios de carga . . . 10
4.2. Repeticiones y aleatorización . . . 12
4.3. Ventanas de medición y sincronización de datos . . . 13
4.4. Protocolo de fallo controlado de un nodo . . . 14
4.5. Plan estadístico preliminar . . . 15

**A. Tablas** **19**
A.1. Matriz de objetivos específicos y requisitos definidos . . . 19

**B. Diagramas** **20**
B.1. Diagrama de arquitectura general . . . 20

**C. Backlog del proyecto** **21**

**D. Enlace a repositorio GitHub** **21**

# 1. Incorporación de la retroalimentación de la Etapa 1

| Observación | Nota | Acción incorporada |
|---|---|---|
| Definición del problema | 9,5/10 | Se diseñó una encuesta de percepción a compañeros de carrera sobre curva de aprendizaje y complejidad de instalación/administración de cada distribución (criterio ya considerado en el proyecto), complementando la literatura secundaria de la Etapa 1; resultados en Etapa 3. |
| Objetivos SMART | 22,5/25 | Se desarrollan los criterios SMART por objetivo específico (OE1–OE4), no solo para el objetivo general (Tabla 2). |
| Análisis de requerimientos | 45/45 | Sin observaciones, se mantiene el mismo rigor en el diseño de Etapa 2. |
| Backlog | Pendiente/20 | Backlog migrado a GitHub Projects y replicado en Google Sheets, con acceso verificado en ambos; tabla-resumen en el Anexo (Tabla 8) para no depender solo del enlace. |

Cuadro 1: Acciones tomadas en respuesta a la retroalimentación de la Etapa 1

| | S | M | A | R | T |
|---|---|---|---|---|---|
| **OE1** | 2 clústeres HA (K3s, MicroK8s), IaC con Terraform/Ansible | 3 nodos `Ready` (kube-state-metrics) | Validado contra el hardware (§2) | Habilita todo el proyecto | Etapa 3 (30 oct) |
| **OE2** | Ejecutar el protocolo experimental completo | Criterios T-08.1–T-08.5 cumplidos por repetición | Calibración piloto ajusta valores preliminares (EP-06) | Responde la pregunta de investigación | Etapa 3–4 |
| **OE3** | Aplicar el plan estadístico (normalidad, prueba, efecto, Holm) | $p$ ajustado y tamaño de efecto por métrica | Depende de OE2; $n$ revisable con evidencia real | Determina si la diferencia es real o azar | Etapa 4 (4 dic) |
| **OE4** | Contrastar con evidencia secundaria de IaaS, con límites explícitos | N.º de fuentes y condiciones documentadas | No requiere acceso a infraestructura cloud real | Contextualiza los hallazgos locales | Etapa 4 (4 dic) |

Cuadro 2: Criterios SMART por objetivo específico

# 2. Diseño de infraestructura virtualizada y red

## 2.1. Topología general

La Figura 1 muestra la topología del laboratorio experimental. El host físico aloja una red bridge virtual sobre la cual se despliegan las tres máquinas virtuales que conforman, en cada ejecución, el clúster de Alta Disponibilidad evaluado. Dado que el laboratorio dispone de un único equipo físico, K3s y MicroK8s no se ejecutan de forma simultánea: cada distribución se aprovisiona, evalúa y destruye de forma independiente sobre la misma topología, reutilizando el mismo rango de direcciones y la misma IP virtual. Esta decisión evita que la energía contabilizada por Scaphandre a nivel de host mezcle el consumo de ambas plataformas dentro de una misma ventana de medición, y refuerza el requisito de homogeneidad experimental (RNF-03) al garantizar que ambos clústeres se despliegan bajo condiciones de red idénticas.

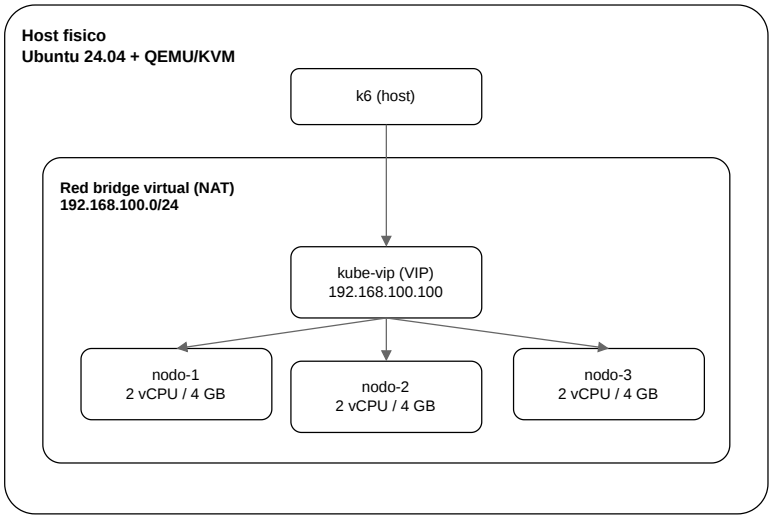



Figura 1: Topología de virtualización del laboratorio experimental

## 2.2. Modo de conectividad de red

Libvirt construye toda red virtual sobre un mismo dispositivo bridge de Linux, independientemente del modo de reenvío configurado, y define tres modos principales: `nat`, `route` e `isolated` (redes sin elemento `forward`). En el modo `nat`, todo el tráfico entre los invitados y la red física se traduce mediante NAT antes de salir del host, las conexiones entrantes desde otras redes quedan prohibidas, mientras que las conexiones entre los propios invitados y entre estos y el host permanecen sin restricción y sin traducción de direcciones. En el modo `route`, el tráfico se reenvía mediante la pila de enrutamiento IP del host sin aplicar NAT, pero la propia documentación de libvirt advierte que «*this presumes that the local LAN router has suitable routing table entries to return traffic to this host*», es decir, requiere que el router físico de la red a la que se conecte el host tenga preconfigurada una ruta estática hacia el rango de la red virtual. Finalmente, en el modo `isolated`, «*the guests can talk to each*

*other, and the host OS, but cannot reach any other machines on the LAN*», es decir, los nodos invitados pueden comunicarse entre sí y con el host, pero quedan completamente aislados de cualquier otro equipo de la red local, incluida la salida a internet (Libvirt Project, 2026).

De estas tres alternativas, `route` queda descartado porque depende de una ruta estática preconfigurada en el router físico externo (Libvirt Project, 2026), fuera del alcance de Terraform y Ansible e incompatible con la reproducibilidad exigida por RNF-01 (el despliegue dejaría de ser reconstruible en una red distinta, por ejemplo si el laboratorio cambia de red). `isolated` también queda descartado, al impedir la salida a internet necesaria para el aprovisionamiento inicial (descarga de paquetes, binario de K3s, snaps de MicroK8s) (Libvirt Project, 2026). Por eliminación, se adopta `nat`, que preserva la reproducibilidad en cualquier red física, habilita la conectividad de salida requerida, y mantiene el rango `192.168.100.0/24` inalcanzable desde el exterior (Libvirt Project, 2026).

Esta decisión es independiente de las variables medidas. Scaphandre y k6 no registran tráfico de red, y aunque Node Exporter sí puede hacerlo mediante su colector `netdev` (Prometheus Authors, 2026c), el rendimiento de red no forma parte de las dimensiones de comparación de este estudio, por lo que esa capacidad no se utiliza. El modo de conectividad se selecciona, en consecuencia, exclusivamente por reproducibilidad y aislamiento.

El procesamiento de *netfilter/conntrack* propio del modo `nat` sí introduce un costo de CPU que Scaphandre contabiliza como energía del host. Sin embargo, se aplica por igual a K3s y MicroK8s, por lo que es una carga simétrica entre ambas condiciones y no una fuente de sesgo (a diferencia de la dependencia externa del modo `route`, que sí comprometería RNF-01).

Las direcciones IP de cada nodo se asignan de forma estática mediante el mecanismo de reserva DHCP por dirección MAC que provee libvirt dentro del elemento `dhcp` de la definición de red, evitando que un nodo cambie de dirección entre una reconstrucción y otra del clúster (Libvirt Project, 2026).

## 2.3. Sistema operativo invitado

Las seis máquinas virtuales del laboratorio (tres por distribución) utilizan Ubuntu Server 24.04 LTS en su variante de imagen cloud como sistema operativo invitado. Ubuntu Server declara oficialmente un requisito mínimo de 1 GB de RAM para esta variante, o 1,5 GB si se instala mediante ISO (Ubuntu Documentation, 2026). Esta decisión se evaluó frente a dos alternativas de menor consumo de recursos:

- **Alpine Linux**: Su sistema de inicio es OpenRC en lugar de systemd (Alpine Linux Wiki, 2026) . MicroK8s se distribuye como paquete snap, y la propia documentación de Canonical sobre los requisitos de Snapcraft exige una distribución GNU/Linux que incluya systemd como requisito de software para el ecosistema de snaps (Canonical, 2026c). Alpine queda, por tanto, excluida como sistema operativo invitado para los nodos de MicroK8s, y adoptarla únicamente para los nodos de K3s introduciría una asimetría de sistema operativo entre ambas distribuciones, incompatible con el requisito de homogeneidad experimental (RNF-03).
- **Debian 12**: Sí utiliza systemd y es, en principio, compatible con snapd. Sin embargo, la documentación oficial de instalación de MicroK8s condiciona su soporte a «*an Ubuntu environment*

*(...) or another operating system which supports snapd»* (Canonical, 2026b), situando a Ubuntu como el entorno validado por el propio proyecto y a cualquier otra distribución como una ruta no verificada oficialmente. Priorizar el entorno explícitamente declarado por el fabricante reduce el riesgo de incompatibilidades no documentadas durante el aprovisionamiento, favoreciendo la reproducibilidad exigida por RNF-01.

Ubuntu Server 24.04 en su variante cloud resulta, en consecuencia, la alternativa que permite utilizar exactamente el mismo sistema operativo base para los nodos de K3s y de MicroK8s, sin introducir pasos de aprovisionamiento asimétricos ni depender de una ruta de instalación no verificada oficialmente por Canonical.

## 2.4. Dimensionamiento de recursos por nodo

Cada nodo virtual se configura con 2 vCPU y 4 GB de RAM. Este dimensionamiento toma como base el requisito mínimo oficial para un nodo servidor de K3s, de 2 núcleos y 2 GB de RAM (K3s Project Authors, 2026), y la recomendación de Canonical de disponer de al menos 4 GB de memoria para ejecutar MicroK8s junto con cargas de trabajo (Canonical, 2026b), adoptando el valor más exigente de ambas fuentes como especificación común a las dos distribuciones.

Sobre las 8 vCPU y 16 GB físicos del host, los tres nodos activos de un clúster consumen 6 vCPU y 12 GB, dejando 2 vCPU y 4 GB reservados para el sistema operativo del host, el generador de carga k6 y las herramientas de recolección de métricas (Scaphandre, Node Exporter). Esta reserva explícita de recursos para el sistema y sus componentes auxiliares sigue el mismo principio que Kubernetes documenta mediante el mecanismo de *Node Allocatable*, que recomienda apartar capacidad de cómputo para los demonios del sistema en lugar de permitir que estos compitan por recursos con las cargas desplegadas (Kubernetes Authors, 2026b). El almacenamiento de cada VM se fija en 50 GB sobre el NVMe de 512 GB del host.

| Recurso | Host físico | Por nodo VM (×3) | Total en uso |
|---|---|---|---|
| CPU | 8 núcleos / 16 hilos | 2 vCPU | 6 de 8 núcleos |
| RAM | 16 GB DDR4 | 4 GB | 12 de 16 GB |
| Disco | 512 GB NVMe | 50 GB (qcow2) | 150 de 512 GB |

Cuadro 3: Asignación de recursos por nodo virtual

La totalidad de los elementos descritos en esta topología, el modo de red, las direcciones estáticas y la definición de cada máquina virtual, se implementará de forma declarativa mediante el proveedor de Terraform `dmacvicar/libvirt`, cuyo recurso `libvirt_network` cubre de forma completa el esquema de red de libvirt (modos de reenvío, bridge, DHCP, VLAN) y cuyo recurso `libvirt_domain` cubre de forma completa el esquema de dominio (dispositivos, CPU, memoria) (dmacvicar, 2026). Esto permite trasladar cada una de las decisiones de diseño descritas en esta sección directamente a código versionable, sin necesidad de scripts imperativos adicionales.

Esta configuración es una estimación inicial, no un valor cerrado. El consumo de memoria de los procesos de interfaz gráfica del host y el de Prometheus (que depende del número de objetivos, la

frecuencia de scrapeo y la cardinalidad de las métricas) no están cuantificados por ninguna fuente oficial aplicable de forma general. Scaphandre, en cambio, declara como objetivo de diseño aproximarse a un «0 overhead» (Hubblo, 2026b), aunque tampoco publica una cifra fija. Por ello, la reserva de 2 vCPU y 4 GB (Tabla 3) se valida empíricamente antes de la campaña definitiva. Se despliega el stack completo junto con k6 en carga alta, monitoreando memoria y swap del host. Si se detecta presión de memoria, la asignación por nodo se reduce a un mínimo de 3 GB, por sobre el mínimo oficial de K3s (2 GB) (K3s Project Authors, 2026), pero con menor margen frente a la recomendación de Canonical para MicroK8s (4 GB, con un piso absoluto de 540 MB) (Canonical, 2026b), por lo que dicha validación se ejecuta de forma independiente para cada distribución, sin asumir que un resultado favorable para una es extrapolable a la otra.

Adicionalmente, al usar el host un entorno de escritorio en lugar de una instalación headless, las ejecuciones definitivas se realizan con la sesión gráfica cerrada, evitando que procesos de interfaz gráfica introduzcan consumo de CPU y energía no controlado en las mediciones de Scaphandre (REX-07).

Durante la implementación se confirmó que la asignación de vCPU en Terraform no garantizaba, por sí sola, aislamiento real frente al resto del sistema. Se implementó entonces afinidad de CPU explícita (*pinning*), fijando cada VM a un núcleo físico completo y reservando uno exclusivo para el host, sin modificar la cantidad de vCPU por nodo (se mantiene en 2, igual para ambas distribuciones). Esto convierte la reserva de recursos en una garantía forzada por el hipervisor, en lugar de una expectativa del planificador, reduciendo variabilidad no controlada entre repeticiones.

## 2.5. Direccionamiento y acceso redundante al plano de control

La red bridge virtual utiliza el rango `192.168.100.0/24`, con direcciones fijas asignadas a cada nodo (.11, .12, .13) y una IP virtual (.100) gestionada por kube-vip en modo ARP. kube-vip anuncia la IP virtual mediante ARP gratuito y determina, mediante elección de líder entre los nodos del clúster, cuál de ellos la aloja en cada momento (kube-vip Authors, 2026a). El propio proyecto documenta dos mecanismos de despliegue distintos según la plataforma de destino, un pod estático en clústeres basados en kubeadm, y un daemonset en K3s y otras distribuciones (kube-vip Authors, 2026c).

Esta decisión reemplaza la propuesta inicial basada en Keepalived/VRRP (Nadas, 2010). A diferencia de Keepalived, un demonio de sistema operativo independiente de Kubernetes, kube-vip se despliega como componente nativo del clúster, reduciendo los procesos que deben automatizarse de forma equivalente entre K3s y MicroK8s. Como contraparte, al ejecutarse como daemonset gestionado por el propio plano de control, la disponibilidad de la IP virtual queda acoplada al estado del proceso de Kubernetes de cada nodo, si ese proceso deja de responder sin que la VM se apague, kube-vip también deja de operar ahí (kube-vip Authors, 2026c). Esta limitación arquitectónica se documenta explícitamente y será contrastada empíricamente en el protocolo de fallo controlado (T-08.4).

Se confirmó una manifestación concreta de esta limitación, específica de MicroK8s: su demonio interno `apiserver-kicker`, que regenera certificados ante cualquier cambio de IP en las interfaces del nodo, no distingue la VIP de kube-vip de una IP primaria real, disparando ciclos de reinicio. Se mitigó deshabilitándolo tras el primer arranque completo del nodo.

# 3. Diseño de arquitectura de observabilidad

## 3.1. Arquitectura general

La Figura 2 resume la arquitectura de observabilidad completa del laboratorio. Cada herramienta cubre exactamente un nivel de observación, evitando solapamientos de responsabilidad. Scaphandre mide energía a nivel de host, Node Exporter caracteriza recursos tanto en el host como en cada nodo virtual, kube-state-metrics observa el estado del clúster a través de la API de Kubernetes, k6 genera y caracteriza la carga desde fuera del clúster, y Prometheus centraliza el almacenamiento y consulta de todas las series temporales excepto las de k6.

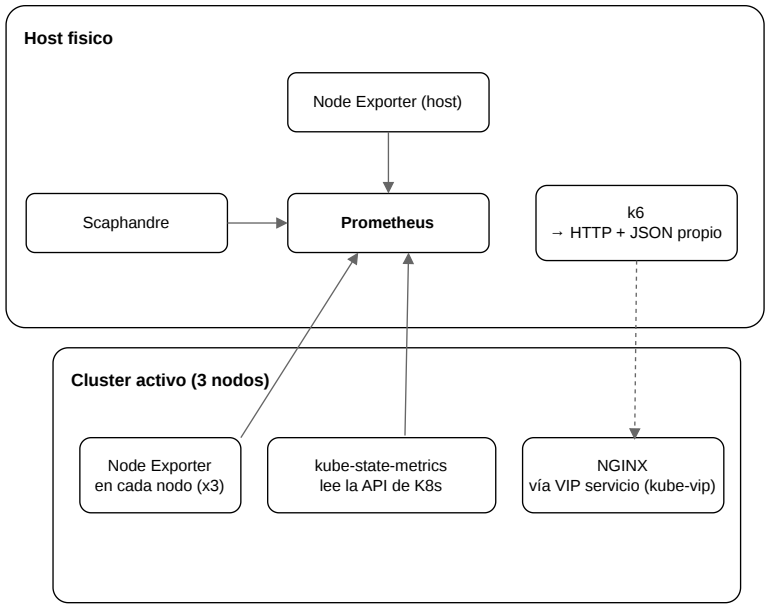



Figura 2: Arquitectura de observabilidad del laboratorio experimental

## 3.2. Medición energética con Scaphandre y RAPL/powercap

Antes de fijar esta decisión de diseño fue necesario verificar su viabilidad sobre el hardware específico del laboratorio, en lugar de asumirla. El AMD Ryzen 7 PRO 5850U del host es una CPU AMD, y la documentación oficial de compatibilidad de Scaphandre distingue explícitamente el soporte de RAPL en hardware Intel del soporte en hardware AMD, para CPUs AMD en bare metal, el sensor PowercapRAPL funciona, pero *«kernel >5.11 required»* (Hubblo, 2026a). Ubuntu 24.04 LTS distribuye por defecto un kernel muy posterior a esa versión mínima, por lo que esta condición se cumple sin necesidad de instalar un kernel alternativo.

Se utilizará el dominio `package`, que agrega el consumo de los núcleos y la parte no asociada a núcleos individuales (*uncore*) de la CPU, por ser el dominio más ampliamente disponible y comparable entre plataformas dentro del propio ecosistema RAPL. La misma tabla de compatibilidad de

Scaphandre indica que, dentro de una máquina virtual, la medición mediante RAPL solo es posible *«if on a qemu/KVM hypervisor that runs scaphandre and the Qemu exporter»* (Hubblo, 2026a), es decir, requeriría desplegar un componente adicional de Scaphandre que monte datos derivados dentro de cada VM. Esta vía se descarta deliberadamente, además del costo de recursos y configuración que añadiría, atribuir energía a nivel de VM contradice el principio metodológico ya establecido de medir exclusivamente a nivel de host y no atribuir directamente esa energía a un nodo virtual específico.

Cada ejecución experimental registrará el valor del contador de energía al inicio y al final de la ventana de medición (RE-02), calculando el incremento correspondiente conforme a la fórmula ya definida en REX-05.

## 3.3. Recolección de CPU y RAM con Node Exporter

Node Exporter se despliega tanto en el host como en cada una de las tres máquinas virtuales del clúster activo, escuchando en el puerto 9100 por defecto (Prometheus Authors, 2026c). Dado el contexto de recursos limitados del laboratorio, no se utilizará la configuración por defecto de Node Exporter, que habilita cerca de veinte colectores. En su lugar, se invoca con la bandera `--collector.disable-defaults`, habilitando explícitamente solo los colectores necesarios para las métricas ya definidas en RO-01 y RO-02: `cpu`, `meminfo`, `diskstats`, `filesystem`, `netdev` y `loadavg`. Este mecanismo de selección explícita de colectores está documentado y puesto a disposición por el propio proyecto (Prometheus Authors, 2026c), y reduce tanto el consumo de CPU del propio exportador como la cardinalidad de series que Prometheus debe almacenar, lo cual es relevante dado que la cardinalidad es, como se estableció en el diseño de infraestructura, el principal factor que determina el consumo de memoria de Prometheus en este laboratorio.

La frecuencia de consulta (scrape) se fija en 10 segundos, valor intermedio entre la resolución temporal necesaria para alinear estos datos con las ventanas de carga de k6 (RO-07) y la reducción de la tasa de ingesta de muestras.

## 3.4. Almacenamiento y consultas en Prometheus

Como se estableció previamente, Prometheus se ejecuta como proceso nativo del host mediante el binario oficial precompilado gestionado por systemd (Prometheus Authors, 2026a), persistiendo su base de series temporales a través de las sucesivas reconstrucciones del clúster.

Para identificar cada conjunto de datos según lo exige RO-08 (distribución, escenario de carga, número de repetición y ventana experimental), se adoptan dos mecanismos complementarios en lugar de uno solo, priorizando nuevamente el ahorro de recursos. La distribución evaluada (`k3s` o `microk8s`) se incorpora como una etiqueta externa (`external_labels`) en la configuración de Prometheus, actualizada y aplicada mediante un reinicio del servicio cada vez que se alterna de distribución, un evento que de todas formas coincide con la reconstrucción completa del clúster, por lo que no introduce una interrupción adicional. El escenario de carga, la repetición y la ventana experimental, en cambio, no se codifican como etiquetas adicionales en cada serie, hacerlo aumentaría la cardinalidad de las métricas y, con ello, el consumo de memoria de Prometheus, exactamente el efecto que se busca evitar. En su lugar, cada ejecución del orquestador experimental registra en un

archivo de metadatos independiente la marca de tiempo de inicio y fin de esa repetición junto con el escenario correspondiente. En la etapa de análisis, estas ventanas se usan para acotar las consultas a Prometheus mediante rangos temporales, siguiendo directamente el mecanismo de verificación de correspondencia temporal (REX-04).

En cuanto a retención, dado que la campaña experimental se extenderá durante varias semanas y el disco del host es un recurso compartido con las máquinas virtuales, no se confía exclusivamente en el período de retención configurado en Prometheus. Al finalizar cada repetición experimental se genera una instantánea (*snapshot*) autocontenida de la base de datos mediante el punto de acceso administrativo `/api/v1/admin/tsdb/snapshot`, habilitado con la bandera `--web.enable-admin-api` (Robust Perception, 2026). Cada instantánea se copia junto con su archivo de metadatos a almacenamiento externo al host, satisfaciendo el requisito de conservar los registros originales de cada ejecución (REX-02) sin depender de que la base de datos en vivo permanezca intacta durante toda la campaña.

## 3.5. Aplicación de referencia

La aplicación de referencia exigida (RF-04) debe cumplir dos condiciones bajo el contexto de recursos limitados de este laboratorio, debe tener una huella de recursos mínima y predecible, para no competir de forma significativa con el propio plano de control de K3s o MicroK8s por los 2 vCPU y 4 GB disponibles en cada nodo, y debe ser deliberadamente simple en términos de cómputo, de manera que las diferencias observadas en energía y utilización de recursos entre ambas distribuciones sean atribuibles a la infraestructura de orquestación y no al comportamiento variable de una lógica de aplicación compleja.

Por esta razón, se descarta diseñar una aplicación con carga computacional propia (por ejemplo, cálculo de números primos u operaciones criptográficas) para diferenciar los escenarios de carga definidos (REX-01). En su lugar, los tres niveles (idle, media, alta) se generan exclusivamente variando la concurrencia y la tasa de peticiones configuradas en k6 (número de VUs y duración), mientras la aplicación de referencia permanece idéntica y con un costo de cómputo mínimo y constante por petición. Esto aísla el comportamiento del plano de datos y del plano de control (programación de peticiones, proxy de servicio, replicación del estado) como la variable de interés principal.

Como aplicación de referencia se utilizará NGINX sirviendo contenido estático mínimo. NGINX utiliza una arquitectura dirigida por eventos en lugar del modelo tradicional de un proceso o hilo por conexión, lo que le permite manejar alta concurrencia con un uso de memoria bajo y predecible (NGINX, Inc., 2026). Al tratarse de una imagen de contenedor pública y sin modificaciones, el mismo manifiesto de Kubernetes (un Deployment de una réplica, con límites de recursos explícitos) se aplica de forma idéntica en ambas distribuciones, satisfaciendo directamente el requisito de una aplicación común (RF-04) sin introducir pasos de construcción o compilación que pudieran divergir entre plataformas.

## 3.6. Mecanismo de exposición: reutilización de kube-vip

Para exponer la aplicación de referencia hacia el generador de carga (que corre en el host, fuera del clúster) se requiere un mecanismo de tipo LoadBalancer. La alternativa más común en clústeres bare-metal, MetalLB, implicaría desplegar y automatizar una herramienta adicional exclusivamente para este propósito. kube-vip, ya presente en el clúster para proveer la IP virtual del plano de control, documenta explícitamente que «*over time it has evolved to incorporate that same functionality for Kubernetes Services of type LoadBalancer*» (kube-vip Authors, 2026b), es decir, el mismo componente ya desplegado para el plano de control puede además anunciar direcciones para servicios de tipo LoadBalancer, utilizando el mismo mecanismo de elección de líder por ARP ya definido para el plano de control (kube-vip Authors, 2026d).

Reutilizar kube-vip para este segundo propósito evita instalar una segunda herramienta de balanceo de carga, ahorrando la memoria y el proceso adicional que MetalLB introduciría en un host donde ya se estableció una reserva de recursos ajustada (Sección 2). Esto exige agregar una segunda IP virtual, distinta de la del plano de control, al rango `192.168.100.0/24` ya definido: se reserva `192.168.100.150` exclusivamente para el Service de tipo LoadBalancer de la aplicación de referencia.

K3s incorpora por defecto un controlador de ingreso (Traefik) y su propio Service LoadBalancer (`ServiceLB`), ausentes en MicroK8s. Ambos se deshabilitan (`--disable traefik --disable servicelb`) para evitar que compitan con kube-vip y mantener la equivalencia de componentes exigida por RNF-03.

## 3.7. Instrumentación de rendimiento con k6

k6 se ejecuta en el host, fuera de las máquinas virtuales, y dirige su carga hacia NGINX como aplicación de referencia, expuesta mediante un Service de tipo LoadBalancer gestionado por kube-vip, según el diseño detallado en la sección de la aplicación de referencia. Sus métricas incorporadas cubren directamente los requisitos ya definidos: número de solicitudes (RO-04), duración de las solicitudes con percentiles p95 y p99 (RO-05), y tasa de solicitudes fallidas (RO-06).

Como se determinó previamente, k6 no transmite sus métricas a Prometheus mediante el módulo experimental de remote write disponible desde la versión 0.42.0 (Grafana Labs, 2026f), dado que su condición de funcionalidad en proceso de estabilización es incompatible con el requisito de comportamiento estable que exige una campaña experimental de varios meses (RNF-01). En su lugar, se utiliza el mecanismo de salida estándar de k6: exportación de resultados granulares en formato JSON mediante `k6 run –out json=resultado.json`, junto con el resumen agregado de fin de ejecución que k6 genera por defecto (Grafana Labs, 2026e). Los resultados de cada repetición se identifican mediante el mismo archivo de metadatos de ventana temporal definido, permitiendo su alineación posterior con las series de Prometheus conforme a RO-07.

## 3.8. Observación de resiliencia y sincronización temporal

Para instrumentar los escenarios de fallo controlado (RR-01 a RR-04) de forma equivalente entre K3s y MicroK8s, se utiliza kube-state-metrics en lugar de instrumentar directamente el almacén de estado

distribuido de cada distribución. kube-state-metrics *«listens to the Kubernetes API server and generates metrics about the state of Kubernetes objects»* (Kubernetes Authors, 2026a), es decir, opera exclusivamente contra la API estándar de Kubernetes, común a ambas distribuciones, evitando así instrumentar el detalle interno específico de etcd o de Dqlite y la asimetría que ello introduciría.

Sobre esta base, se definen operacionalmente los tres momentos exigidos por los requisitos de resiliencia:

- **Inicio del fallo (RR-02)**: la marca de tiempo registrada por el propio script de inyección de fallos al ejecutar la acción correspondiente (apagado de la VM, terminación del proceso de K3s o MicroK8s, o corte de la interfaz de red).
- **Detección del fallo (RR-02)**: el instante en que la métrica `kube_node_status_condition` expuesta por kube-state-metrics refleja la condición `Ready` en estado falso para el nodo afectado.
- **Recuperación (RR-03)**: se reportan dos criterios de forma independiente en lugar de uno solo, dado que capturan dimensiones distintas del problema, a nivel de infraestructura, el retorno de la misma métrica de kube-state-metrics a la condición `Ready` en estado verdadero, y a nivel de aplicación, la normalización de la tasa de error reportada por k6 (`http_req_failed`) a su valor previo al fallo.

Finalmente, la sincronización temporal entre todas las fuentes (RO-07) depende de que los relojes del host y de las seis máquinas virtuales no diverjan de forma apreciable durante una ventana experimental. Para ello, se configurará `chrony` tanto en el host como en cada VM, con el host actuando como fuente de referencia local para las VMs, dado que el propio host es además donde residen Prometheus y Scaphandre y, por tanto, la referencia temporal contra la cual se comparan el resto de las fuentes.

# 4. Diseño del protocolo experimental

## 4.1. Escenarios de carga

Para generar los tres escenarios de carga (idle, media y alta) se utiliza el ejecutor `ramping-arrival-rate` de k6, que implementa un modelo abierto de programación de carga. Las iteraciones se inician a una tasa objetivo fija, independiente del tiempo que tome completar la iteración anterior (Grafana Labs, 2026c). Esta decisión no es casual, en el modelo cerrado (usado por ejecutores como `ramping-vus` o `constant-vus`), la propia documentación de k6 advierte que *«the target system's response time can influence the throughput of the test»* (Grafana Labs, 2026c), es decir, el tiempo que tarde la plataforma en responder puede alterar directamente cuánta carga efectiva llega a aplicarse durante la prueba, incluso manteniendo fija la configuración de VUs. Dado que el objetivo central de este trabajo es comparar el overhead de dos plataformas que pueden tener tiempos de respuesta distintos, un modelo cerrado aplicaría de forma automática menos carga real a la plataforma más lenta, introduciendo un sesgo que contradice el principio de homogeneidad experimental (RNF-03). El modelo abierto garantiza que ambas distribuciones reciban exactamente la misma tasa de

solicitudes objetivo, dejando que la tasa de error y la latencia (no la cantidad de carga aplicada) sean las variables que reflejen las diferencias entre plataformas.

Cada escenario de carga (salvo idle) se estructura en tres fases mediante el ejecutor `ramping-arrival-rate` (Grafana Labs, 2026d): una fase de ascenso, una fase estable, y una fase de descenso. Esta estructura corresponde a una práctica de diseño de perfiles de carga documentada por el propio proyecto k6, donde la fase de ascenso evita picos de arranque en frío, la fase estable es la única ventana considerada válida para la recolección de métricas, y la fase de descenso permite un cierre ordenado de las conexiones activas (Grafana Labs, 2026b). Este es precisamente el criterio de estabilización exigido, únicamente la fase estable de cada escenario se utiliza como ventana de medición para Scaphandre, Node Exporter y Prometheus. Los datos de las fases de ascenso y descenso se descartan del análisis por representar un estado transitorio y no comparable entre repeticiones.

| Escenario | Tasa objetivo | Ascenso | Estable | Descenso |
|---|---|---|---|---|
| Idle | Sin tráfico de k6 | — | 3 min | — |
| Media | 50 % de la tasa de saturación (T-06.2) | 30 s | 3 min | 30 s |
| Alta | Inmediatamente bajo la tasa de saturación | 30 s | 3 min | 30 s |

Cuadro 4: Escenarios de carga (tasas sujetas a calibración piloto)

El valor concreto de la tasa objetivo para los escenarios media y alta no se fija de antemano como una cifra arbitraria, sino que se calibra empíricamente contra el presupuesto de recursos ya establecido en el diseño de infraestructura (Sección 2). Dado que NGINX sirviendo contenido estático puede sostener tasas de solicitudes muy superiores a las que un nodo de 2 vCPU puede procesar en conjunto con el resto de sus procesos, el factor limitante de interés no es la capacidad de la aplicación, sino la capacidad del nodo completo que la aloja, incluyendo kubelet, kube-proxy y, si corresponde, los componentes del plano de control. Durante la calibración piloto (EP-06), se incrementará gradualmente la tasa objetivo mientras se observa, mediante Node Exporter, la utilización de CPU del nodo que aloja el pod de NGINX en un momento dado. Se define «carga alta» como la tasa objetivo inmediatamente inferior a aquella en la que dicho nodo se aproxima a la saturación de sus 2 vCPU, dejando un margen de seguridad que evite inducir una caída del nodo por agotamiento de recursos, un evento distinto y no controlado que debe permanecer separado del escenario de fallo controlado definido (T-08.4). «Carga media» se fija, como punto de partida, en la mitad de la tasa así determinada para carga alta.

Esta misma calibración exige revisar los límites de recursos asignados al propio pod de NGINX, un límite de CPU en Kubernetes se traduce en una cuota de tiempo de CPU aplicada mediante el planificador CFS del kernel (*Completely Fair Scheduler*) sobre un período fijo, de modo que un contenedor puede ser detenido por el kernel al agotar su cuota dentro de ese período incluso cuando el nodo dispone de capacidad de CPU ociosa (Kubernetes Authors, 2023). Si a NGINX se le asignara un límite de CPU demasiado restrictivo, este mecanismo limitaría artificialmente las solicitudes procesadas antes de que el nodo alcance su propia saturación, y lo que se mediría sería la restricción impuesta al contenedor y no el comportamiento de la plataforma de orquestación. En consecuencia, el límite

de CPU de NGINX se fija en un valor deliberadamente holgado (por ejemplo, 500m), de modo que el techo real de capacidad sea el del nodo completo y no el del contenedor individual.

Es posible que NGINX estático no exija al nodo de forma realista ni a tasas altas, dado lo eficientes que son NGINX y kube-proxy para ese tráfico. Si la calibración piloto lo confirma, la contingencia es agregar al endpoint un costo computacional mínimo y determinístico (por ejemplo, un hash SHA-256 repetido un número fijo de veces), preservando la equivalencia entre distribuciones sin depender de tasas de solicitud poco prácticas.

Por último, el generador de carga se configura con `discardResponseBodies: true` (no se necesita el cuerpo de las respuestas) y con un techo explícito de VUs (`maxVUs`). k6 solo activa los VUs necesarios para sostener la tasa objetivo, cantidad que se mantiene baja con tiempos de respuesta breves (Grafana Labs, 2026a). Sin ese techo, si el nodo se degradara y las respuestas se hicieran más lentas, k6 generaría más VUs para compensar, consumiendo memoria adicional justo en el momento de mayor presión.

El escenario idle no genera tráfico de k6 en absoluto: su propósito es capturar el consumo base del clúster (plano de control, kube-vip, Node Exporter, kube-state-metrics) sin carga externa aplicada, sirviendo como línea base frente a la cual interpretar los otros dos escenarios.

## 4.2. Repeticiones y aleatorización

El diseño de las repeticiones debe resolver dos preguntas relacionadas pero distintas: cuántas veces se repite cada combinación de distribución y escenario, y en qué orden se ejecutan esas repeticiones para que las condiciones no controladas del laboratorio (incluye térmica del host, variabilidad de fondo a lo largo de una sesión larga de varias horas) no se confundan con la variable de interés, que es la distribución evaluada.

Para la segunda pregunta se adopta el principio de diseño experimental formulado por el NIST: *«block what you can, randomize what you cannot»* (NIST/SEMATECH, 2026), es decir, los factores que pueden fijarse deliberadamente se controlan mediante bloques, y solo los factores que escapan al control del experimentador se dejan sujetos a variabilidad aleatoria. La misma documentación advierte que las repeticiones de un experimento no deberían ejecutarse bajo condiciones completamente homogéneas cuando existen factores de tiempo o de entorno que pueden introducir una deriva sistemática, y recomienda organizar dichas repeticiones en bloques que distribuyan esa deriva de forma equitativa entre los tratamientos comparados, en lugar de dejarla concentrada en uno de ellos (NIST/SEMATECH, 2026).

Aplicado a este laboratorio, el factor no controlable más relevante es el tiempo: una sesión de varias horas puede implicar deriva térmica del hardware o variaciones del sistema operativo del host. Si todas las repeticiones de K3s se ejecutaran antes que las de MicroK8s (o viceversa), esa deriva quedaría confundida con la distribución evaluada. Por ello, las repeticiones se organizan en dos bloques con el orden invertido (K3s $\rightarrow$ MicroK8s en el primero, MicroK8s $\rightarrow$ K3s en el segundo): este diseño contrabalanceado hace que la deriva afecte por igual a ambas distribuciones, en vez de favorecer a la que se ejecuta primero.

Dentro de cada repetición, en cambio, los tres escenarios de carga (idle, media y alta) se ejecutan

siempre en el mismo orden creciente. Esta decisión no contradice el principio de aleatorización, el orden idle, media, alta no es en sí mismo una variable de comparación entre K3s y MicroK8s, sino una rampa de intensidad aplicada de forma idéntica a ambas distribuciones en cada repetición. Aleatorizar este orden introduciría, además, una complejidad operativa considerable sin un beneficio claro, ya que el escenario idle está pensado explícitamente como línea base de referencia y su interpretación depende de que preceda a los escenarios con carga, no de que se intercale aleatoriamente entre ellos.

Respecto de la reconstrucción del clúster, se distingue entre transiciones que implican un cambio real de distribución y la transición intermedia entre los dos segmentos consecutivos de MicroK8s. Las primeras (K3s a MicroK8s, y MicroK8s a K3s) requieren necesariamente un ciclo completo de `terraform destroy` seguido de `terraform apply` y Ansible, dado que el software instalado debe cambiar. La transición intermedia, en cambio, no involucra un cambio de software, las tres máquinas virtuales se apagan y se vuelven a encender, preservando su disco y evitando así repetir la instalación del sistema operativo base y el bootstrap del clúster, la parte más costosa en tiempo de un despliegue completo. Tras el reinicio, se verifica mediante kube-state-metrics que los tres nodos retornen al estado `Ready` y que el quórum del almacén de estado distribuido se reforme correctamente antes de iniciar la siguiente repetición, extendiendo el mismo criterio de estabilización ya exigido por REX-03.

Esto tiene una consecuencia: si el tiempo de despliegue se midiera contando los ciclos de esta secuencia, MicroK8s quedaría con menos muestras que K3s (solo este acumula dos despliegues completos reales). Para evitarlo, el tiempo de despliegue se mide aparte, como experimento propio. Durante la calibración piloto (EP-06), se ejecutan series independientes de `terraform destroy/apply` para cada distribución, cronometrando sin carga de k6. Así, la secuencia de bloques queda libre para usar el mecanismo más rápido en cada transición sin afectar ninguna métrica.

| | **Bloque 1** | **Bloque 2** |
|---|---|---|
| Orden de evaluación | K3s → MicroK8s | MicroK8s → K3s |
| Repeticiones por distribución | 5 | 5 |

Cuadro 5: Plan de bloques contrabalanceado (10 repeticiones totales por distribución)

El valor de 5 repeticiones por bloque (10 en total por distribución) se adopta como punto de partida y no como un valor cerrado, al tratarse de una cifra preliminar, deberá ajustarse de acuerdo con la variabilidad observada durante la calibración piloto (EP-06) y con el análisis de potencia estadística (T-08.5), de manera que el número final de repeticiones responda a evidencia empírica sobre la dispersión de los datos y no a una elección arbitraria.

## 4.3. Ventanas de medición y sincronización de datos

La ventana de medición válida para cada repetición corresponde exclusivamente a la fase estable de cada escenario (T-08.1): las marcas de tiempo de inicio ($t_{inicio}$) y fin ($t_{fin}$) de esa fase, y no las de todo el escenario, son las que se registran en el archivo de metadatos ya definido en el diseño de observabilidad (Sección 3), junto con la distribución y el escenario correspondientes.

Estas dos marcas de tiempo son el mecanismo único de alineación entre las tres fuentes de datos:

- **Scaphandre**: los valores de energía inicial y final exigidos por RE-02 se obtienen mediante una consulta puntual a Prometheus en $t_{inicio}$ y $t_{fin}$, tomando la muestra disponible más cercana a cada marca, dado el intervalo de scrapeo de 10 segundos ya definido. El incremento energético se calcula conforme a la fórmula de REX-05.
- **Node Exporter**: las métricas de CPU y memoria del host y de los nodos se extraen mediante una consulta de rango a Prometheus acotada al intervalo $[t_{inicio}, t_{fin}]$.
- **k6**: del archivo JSON de resultados se conservan únicamente las iteraciones cuya marca de tiempo cae dentro de $[t_{inicio}, t_{fin}]$, descartando explícitamente las correspondientes a las fases de ascenso y descenso del escenario, en consistencia con el criterio de estabilización ya definido en T-08.1.

Antes de aceptar una repetición como válida, se verifica que no existan interrupciones de recolección durante la ventana, conforme exige REX-04. Para ello se consulta la serie `up`, que Prometheus almacena automáticamente por cada objetivo scrapeado y que toma el valor 1 *«if the instance is healthy, i.e. reachable»* o 0 si el scrape falló (Prometheus Authors, 2026b). Si `up{job="node_exporter"}` o `up{job="scaphandre"}` toma el valor 0 en algún punto dentro de $[t_{inicio}, t_{fin}]$, la repetición se descarta y se repite, en lugar de aceptarse con datos potencialmente incompletos.

La sincronización de relojes entre las fuentes (host y VMs) mediante `chrony` ya fue establecida en el diseño de observabilidad (Sección 3) y constituye el requisito de base que hace válida esta alineación por marcas de tiempo, sin relojes razonablemente sincronizados, $t_{inicio}$ y $t_{fin}$ registrados por el orquestador no corresponderían al mismo instante real en cada fuente.

## 4.4. Protocolo de fallo controlado de un nodo

El fallo controlado exigido por RR-01 se inyecta específicamente mediante la terminación del proceso de orquestación de un nodo, sin apagar la máquina virtual ni interrumpir su conectividad de red. Esta elección no es arbitraria, es precisamente el escenario que permite contrastar empíricamente la limitación arquitectónica de kube-vip ya documentada en el diseño de infraestructura (Sección 2), donde se estableció que la disponibilidad de la IP virtual queda acoplada al estado del proceso de Kubernetes de cada nodo. Los escenarios de apagado completo de la VM y de corte de la interfaz de red se documentan como extensiones de menor prioridad, dado que el foco es específicamente ese acoplamiento, y no una caracterización exhaustiva de todos los modos de fallo posibles.

**Selección del nodo**: se falla el nodo que aloja la IP virtual de kube-vip en el momento de la inyección, identificado consultando la elección de líder antes de ejecutar el fallo. Esto garantiza que el escenario efectivamente exija una reelección de líder y una migración de la IP virtual, y no el fallo de un nodo secundario cuya caída no afecta el punto de acceso al plano de control.

**Momento de inyección**: el fallo se inyecta durante la fase estable del escenario de carga media (REX-01), no de idle ni de alta. Idle queda descartado porque RR-04 exige medir el impacto sobre solicitudes en curso, lo que requiere tráfico real. Alta queda descartada porque, al operar cerca del punto

de saturación del nodo, cualquier efecto del fallo se confundiría con el propio comportamiento del nodo bajo saturación.

**Mecanismo de inyección**: se termina abruptamente el proceso mediante `systemctl kill -s SIGKILL k3s` en el caso de K3s. Para MicroK8s, el proceso equivalente desde la versión 1.21 es `snap.microk8s.daemon-kubelite`, que consolida en un único proceso al servidor de API, el programador, el gestor de controladores, el kubelet y el proxy (Canonical, 2026a); el fallo se inyecta, por tanto, mediante `systemctl kill -s SIGKILL snap.microk8s.daemon-kubelite`. En ambos casos, el sistema operativo, la red y el disco de la VM continúan funcionando con normalidad, únicamente el proceso de orquestación deja de responder, aislando el fallo al componente de interés.

**Cronología de la ventana de fallo**:

| Momento | Definición |
|---|---|
| $t_{pre}$ | Inicio de un período breve de tráfico normal previo al fallo (línea base de error) |
| $t_{fallo}$ (RR-02) | Marca de tiempo registrada por el script al ejecutar `systemctl kill` |
| $t_{deteccion}$ (RR-02) | Transición de `kube_node_status_condition` a `Ready=false` |
| $t_{recuperacion}$ (RR-03) | Doble criterio: retorno a `Ready=true` (infraestructura) y normalización de `http_req_failed` (aplicación), ya definidos en T-07.5 |
| $t_{post}$ | Período de observación posterior, para confirmar que la recuperación es sostenida y no un falso positivo transitorio |

Cuadro 6: Cronología del protocolo de fallo controlado

**Restauración**: una vez completado $t_{post}$, se reinicia el proceso de orquestación en el nodo afectado (`systemctl start k3s` o el equivalente de MicroK8s), y se aplica el mismo criterio de estabilización de REX-03 antes de iniciar la siguiente repetición.

**Seguridad del quórum**: en cada inyección se falla un único nodo de los tres que componen el clúster, preservando la mayoría (2 de 3) tanto en etcd como en Dqlite. Esto respeta explícitamente el criterio de seguridad ya definido para los escenarios de fallo controlado (RR-01): el protocolo evita provocar simultáneamente una cantidad de fallos que implique la pérdida intencional del quórum requerido por la arquitectura.

**Repeticiones**: dado que cada repetición de este protocolo requiere esperar la recuperación completa antes de continuar, resultando en ciclos más largos que los de los escenarios de carga puros, se adopta un número menor de repeticiones como punto de partida (5 por distribución), igualmente sujeto a ajuste en la calibración piloto (EP-06).

## 4.5. Plan estadístico preliminar

El análisis estadístico definitivo se determinará según el comportamiento real de los datos, pero es posible fijar de antemano el procedimiento de decisión que se aplicará, en lugar de dejarlo abierto hasta la etapa de análisis.

**Medidas descriptivas**: para cada métrica cuantitativa (energía contabilizada, potencia media, energía por solicitud válida, utilización de CPU y memoria, throughput, latencia con percentiles p95/p99, tasa de error, tiempo de detección, tiempo de recuperación y solicitudes afectadas) se reportará la mediana y el rango intercuartílico, además de la media y la desviación estándar. La mediana y el rango intercuartílico se priorizan como medidas principales de tendencia central y dispersión, dado que son más robustas frente a valores atípicos que podrían surgir de eventos puntuales del sistema (por ejemplo, una recolección de basura del runtime o una interrupción breve del scrape), y dado el tamaño muestral reducido (5 a 10 repeticiones por combinación) que hace a la media particularmente sensible a esos valores atípicos.

**Verificación de supuestos**: antes de seleccionar una prueba de hipótesis, se evalúa la normalidad de cada distribución de datos mediante el test de Shapiro y Wilk (1965), apropiado para muestras pequeñas como esta. La homogeneidad de varianza entre K3s y MicroK8s se evalúa de forma complementaria cuando la normalidad no se rechaza, ya que condiciona la elección entre una prueba t de Student clásica y su variante de Welch.

**Criterio de selección de la prueba**: si ambas distribuciones no se apartan significativamente de la normalidad y presentan varianzas homogéneas, se utiliza una prueba t de Student para muestras independientes, si la normalidad se cumple pero las varianzas difieren, se utiliza la prueba t de Welch. Si el supuesto de normalidad se rechaza para alguna de las dos distribuciones, el escenario más probable dado el tamaño muestral reducido y la naturaleza de las métricas de sistemas, donde son comunes las distribuciones asimétricas y con colas largas, se utiliza en su lugar la prueba no paramétrica de Mann y Whitney (1947), que no exige el supuesto de normalidad y compara las distribuciones mediante sus rangos.

**Tamaño del efecto**: independientemente de la prueba seleccionada, la significancia estadística no se reportará de forma aislada. Se complementa con una medida de tamaño del efecto, la delta de Cliff (1993) en el caso no paramétrico, por ser la medida de efecto naturalmente asociada a la prueba de Mann y Whitney (1947), o $d$ de Cohen en el caso paramétrico. Esto permite distinguir una diferencia estadísticamente significativa pero de magnitud práctica despreciable de una diferencia que además sea relevante para la comparación entre distribuciones.

**Corrección por comparaciones múltiples**: dado que se evalúan numerosas métricas de forma simultánea entre las mismas dos distribuciones, la probabilidad de obtener al menos un resultado significativo por azar aumenta con el número de pruebas realizadas. Para controlar esta inflación del error de tipo I, los valores p se ajustan mediante el método secuencial de Holm (1979), que ofrece mayor potencia estadística que la corrección de Bonferroni clásica sin sacrificar el control del error familiar.

**Revisión del tamaño muestral**: los valores de repeticiones propuestos en T-08.2 (10 por distribución para los escenarios de carga) y en T-08.4 (5 por distribución para el fallo controlado) son preliminares. Una vez completada la calibración piloto (EP-06) y obtenida una estimación real de la variabilidad de cada métrica, se revisará si estos tamaños muestrales ofrecen potencia estadística suficiente para detectar diferencias de magnitud relevante entre K3s y MicroK8s, ajustándolos si la evidencia empírica lo justifica.

# Referencias


Mann, H. B., & Whitney, D. R. (1947). On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*, *18*(1), 50-60. https://doi.org/10.1214/aoms/1177730491

Shapiro, S. S., & Wilk, M. B. (1965). An Analysis of Variance Test for Normality (Complete Samples). *Biometrika*, *52*(3/4), 591-611. https://doi.org/10.1093/biomet/52.3-4.591

Holm, S. (1979). A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, *6*(2), 65-70.

Cliff, N. (1993). Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions. *Psychological Bulletin*, *114*(3), 494-509. https://doi.org/10.1037/0033-2909.114.3.494

Nadas, S. (2010, marzo). *Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6* [RFC 5798, Internet Engineering Task Force]. https://www.rfc-editor.org/rfc/rfc5798.html

Kubernetes Authors. (2023). *The Case for Kubernetes Resource Limits*. https://kubernetes.io/blog/2023/11/16/the-case-for-kubernetes-resource-limits/

Alpine Linux Wiki. (2026). *OpenRC*. https://wiki.alpinelinux.org/wiki/Openrc

Canonical. (2026a). *Configuring MicroK8s services*. https://canonical.com/microk8s/docs/configuring-services

Canonical. (2026b). *Get started | MicroK8s Documentation*. https://canonical.com/microk8s/docs/getting-started

Canonical. (2026c). *System requirements | Snapcraft documentation*. https://ubuntu.com/docs/snapcraft/9/reference/system-requirements/

dmacvicar. (2026). *terraform-provider-libvirt (README)*. https://github.com/dmacvicar/terraform-provider-libvirt

Grafana Labs. (2026a). *Arrival-rate VU allocation | Grafana k6 documentation*. https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/arrival-rate-vu-allocation.md

Grafana Labs. (2026b). *Designing a Realistic Load Profile | Grafana Learning Hub*. https://grafana.com/docs/learning-hub/k6-performance-testing/03-establishing-a-baseline/17-designing-load-profile/

Grafana Labs. (2026c). *Open and closed models | Grafana k6 documentation*. https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/

Grafana Labs. (2026d). *Ramping VUs | Grafana k6 documentation*. https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/ramping-vus/

Grafana Labs. (2026e). *Results output | Grafana k6 documentation*. https://grafana.com/docs/k6/latest/get-started/results-output

Grafana Labs. (2026f). *xk6-output-prometheus-remote (README)*. https://github.com/grafana/xk6-output-prometheus-remote

Hubblo. (2026a). *Compatibility | Scaphandre documentation*. https://hubblo-org.github.io/scaphandre-documentation/compatibility.html

Hubblo. (2026b). *Scaphandre: energy consumption metrology agent (README)*. https://github.com/hubblo-org/scaphandre

K3s Project Authors. (2026). *Requirements | K3s Documentation*. https://docs.k3s.io/installation/requirements

Kubernetes Authors. (2026a). *kube-state-metrics (README)*. https://github.com/kubernetes/kube-state-metrics

Kubernetes Authors. (2026b). *Reserve Compute Resources for System Daemons*. https://kubernetes.io/docs/tasks/administer-cluster/reserve-compute-resources/

kube-vip Authors. (2026a). *ARP | kube-vip documentation*. https://kube-vip.io/docs/modes/arp

kube-vip Authors. (2026b). *Features | kube-vip documentation*. https://kube-vip.io/docs/about/features/

kube-vip Authors. (2026c). *kube-vip: High Availability and Load-Balancing (README)*. https://github.com/kube-vip/kube-vip

kube-vip Authors. (2026d). *Kubernetes Load-Balancer service | kube-vip documentation*. https://kube-vip.io/docs/usage/kubernetes-services/

Libvirt Project. (2026). *Network XML format*. https://libvirt.org/formatnetwork.html

NGINX, Inc. (2026). *NGINX Wiki*. https://www.nginx.com/resources/wiki/

NIST/SEMATECH. (2026). *Randomized Block Designs | e-Handbook of Statistical Methods*. https://itl.nist.gov/div898/handbook/pri/section3/pri332.htm

Prometheus Authors. (2026a). *Download | Prometheus*. https://prometheus.io/download/

Prometheus Authors. (2026b). *Jobs and instances | Prometheus documentation*. https://prometheus.io/docs/concepts/jobs_instances/

Prometheus Authors. (2026c). *node_exporter: Exporter for machine metrics (README)*. https://github.com/prometheus/node_exporter

Robust Perception. (2026). *Taking Snapshots of Prometheus Data*. https://www.robustperception.io/taking-snapshots-of-prometheus-data

Ubuntu Documentation. (2026). *System requirements*. https://documentation.ubuntu.com/server/reference/installation/system-requirements

# A. Tablas

## A.1. Matriz de objetivos específicos y requisitos definidos

| **Objetivo** | **Requisitos asociados** | **Cobertura** |
|---|---|---|
| OE1 | RF-01, RF-02, RF-03, RF-04, RF-05, RNF-01, RNF-02, RNF-03, RNF-04 | Implementación de ambos clústeres HA, acceso redundante al plano de control y automatización reproducible de la infraestructura mediante Terraform y Ansible. |
| OE2 | RF-04, RNF-03, RE-01, RE-02, RE-03, RE-04, REX-01, REX-02, REX-03, REX-04, REX-05, REX-07, RR-01, RR-02, RR-03, RR-04 | Diseño y ejecución del protocolo experimental: medición energética del host, escenarios de carga, réplicas, calidad de datos y escenario de fallo controlado. |
| OE3 | REX-02, REX-04, REX-05, REX-06, RR-02, RR-03, RR-04 | Análisis estadístico de las diferencias observadas entre K3s y MicroK8s, incluyendo los indicadores de recuperación ante fallos. |
| OE4 | RC-01, RC-02, RC-03 | Contraste contextual de los resultados locales con evidencia secundaria de infraestructura IaaS en nube pública. |

Cuadro 7: Matriz entre los objetivos específicos y los requisitos definidos

# B. Diagramas

## B.1. Diagrama de arquitectura general

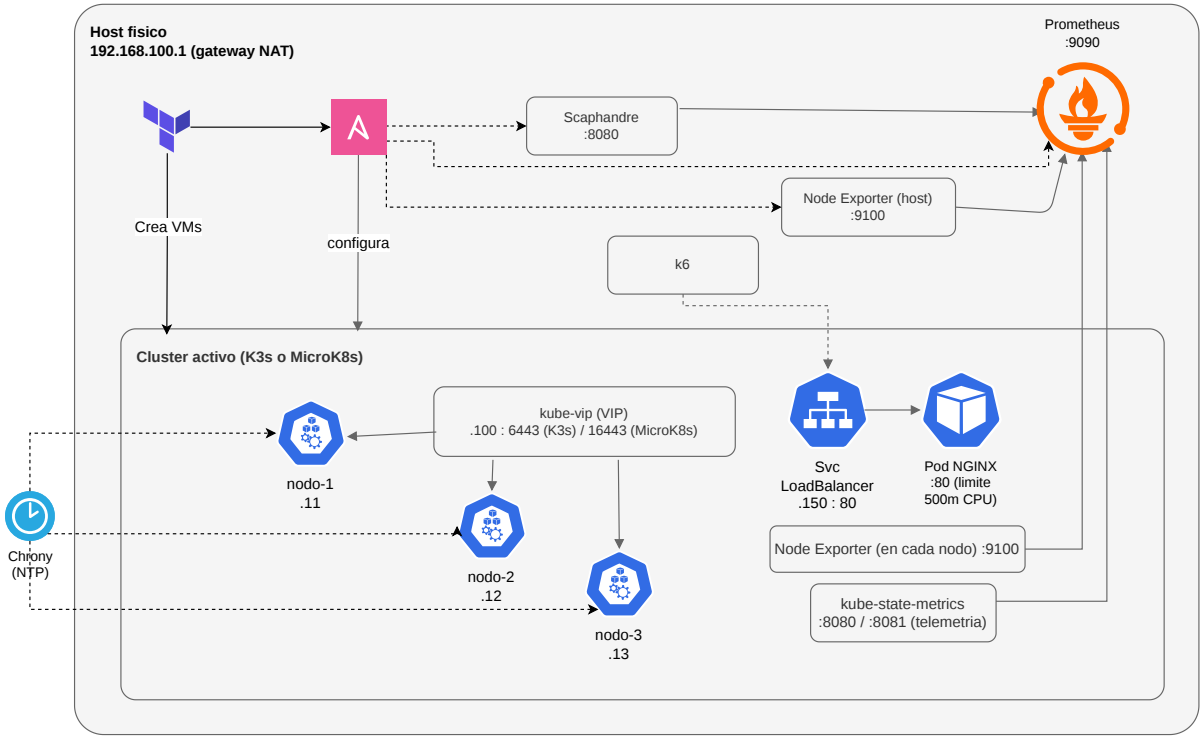



Figura 3: Diagrama de arquitectura del laboratorio experimental

# C. Backlog del proyecto

El backlog se gestiona en GitHub Projects. Cada épica, historia y tarea es un *issue* del repositorio: la jerarquía se representa mediante *sub-issues*, cada etapa es un *milestone* con su fecha de evaluación, y el tipo, los objetivos (`og`, `oe1`–`oe4`) y la prioridad se registran como *labels*. La Tabla 8 resume su estado al cierre de la Etapa 2.

| Épica | Etapa | Descripción | Hist. | Tareas | Realiz. | Horas est. |
|---|---|---|---|---|---|---|
| EP-01 | 1 | Definición, investigación y análisis de requerimientos | 5 | 20 | 20 | 74 |
| EP-02 | 2 | Diseño experimental y técnico | 4 | 17 | 17 | 40 |
| EP-03 | 2 | Diseño e incorporación de retroalimentación | 1 | 2 | 2 | 2 |
| EP-04 | 2 | Implementación de infraestructura y Alta Disponibilidad | 5 | 24 | 24 | 32 |
| EP-05 | 3 | Implementación de observabilidad y aplicación | 4 | 15 | 0 | 29 |
| EP-06 | 3 | Pruebas iniciales, mejoras y validación preliminar | 4 | 14 | 0 | 47 |
| EP-07 | 4 | Campaña experimental, análisis y validación final | 4 | 13 | 0 | 62 |
| EP-08 | 4 | Redacción final, conclusiones y defensa | 4 | 11 | 0 | 60 |
| **Total** | | | **31** | **116** | **63** | **346** |

Cuadro 8: Resumen del backlog por épica (estado al cierre de la Etapa 2)

Las horas estimadas por etapa son 74 (Etapa 1), 74 (Etapa 2), 76 (Etapa 3) y 122 (Etapa 4). El 100 % de las tareas de las Etapas 1 y 2 (63 de 63) está realizado y validado contra el repositorio.

Project: https://github.com/users/Coto2412/projects/2
Backlog (Google Sheets): https://docs.google.com/spreadsheets/d/1BBvxy5O2bchPX32bLfcvAZIxatpPCwSWgb9XDpb6u4g/edit?usp=sharing

# D. Enlace a repositorio GitHub

https://github.com/Coto2412/k3s-microk8s-ha-benchmarks.git